Credit-Risk Model with a CI/CD Quality Gate

Elective II: Machine Learning Operations (Project 1) — Instructor: Dr. Santosh Kumar — NMIMS NSOMASA
Group 13 (Sheet 2): Nihar Sahu (B010) & Shaun Shaju (B012) — 24 October 2026

Summary. We built a versioned, config-driven, tested and gated training workflow for a credit-card default model (800 customers in v1, 1,100 in v2; 18.9% / 19.0% high-risk). The repository holds 13 commits, 3 YAML environments, 17 tests, a GitHub Actions gate, 12 tracked MLflow runs (plus drill and audit runs) and a registered XGBoost champion. Central results: accuracy is useless here (Spearman with recall -0.80); the v2 gain in PR-AUC ($+0.007$, Welch $p = 0.8378$) is *not* distinguishable from seed noise (SD 0.0523); the composite gate (Rule C) accepts 7 of 12 runs; and the Income_Bracket audit shows no detectable disparity but has too few defaults to prove parity. We independently re-computed the data statistics, the baseline run and all tests of significance, and we list the weaknesses we found in Sec. 9 instead of hiding them.

1 Lifecycle Map (Task 1)

DevOps	MLOps activity	Concrete artefact / workflow step
Plan	Problem, metric, thresholds	`config/*.yaml`, `baseline_metrics.json` (PR-AUC, recall floor, gate tolerances)
Code	Features, models	`src/feature_engineering.py`, `train.py`, `evaluate.py`; scripts in `scripts/`
Build	Validated pipeline	`config_schema.py` (Pydantic V2), `data_loader.py` (SHA-256 check), `requirements.txt`
Test	Unit tests + statistical gate	`tests/` (17 pytest tests), `src/gate.py`, `.github/workflows/ci.yml`
Release	Registry	MLflow (SQLite) model `CreditRiskGateModel`; run tags carry source commit; config YAML stored per run
Deploy	Serving prep	`config/prod.yaml` (v2, 150 trees); registered pipeline artefact (no live endpoint)
Operate	Retraining on new data	`scripts/run_v1_v2_drill.py` (v2 arrives $\Rightarrow$ retrain through same workflow)
Monitor	Error audit	`scripts/run_fairness_defense.py`: subgroup errors by Income_Bracket

ML differs from ordinary delivery in three ways. (i) *Dual dependency:* behaviour is a function of code *and* data: with code frozen, moving from v1 to v2 changes the model, so data needs a version identifier just like code. (ii) *Probabilistic builds:* rebuilding with a different seed changes the artefact: the same XGBoost configuration on v1 gives PR-AUC 0.7883 / 0.8926 / 0.8346 for seeds 42 / 101 / 777 (SD = 0.0523); our independent logistic-regression check over the same seeds gives SD ≈ 0.059. A gate must therefore reason about noise, which a compiler never needs to. (iii) *Tests of correctness are not tests of quality:* a configuration with 2 trees and depth 1 passes all pytest tests but is a bad model (PR #102), so the CI needs a statistical gate besides unit tests. **Intentionally excluded:** a live serving endpoint, production monitoring/drift detection, a remote artefact store (MLflow runs on local SQLite), automated hyper-parameter search, and the human model-risk approval step (modelled only as a documented policy field).

2 Repository and Data Versioning (Task 2)

The layout (`config/, data/, src/, scripts/, tests/, reports/, .github/`) is documented in the README. `data/data_manifest.json` stores file name, shape and SHA-256 of both datasets (v1: `91c12c5c...897237`, 800×10; v2: `e9514b4f...ca1c09`, 1100×10); `load_data` recomputes the hash and raises `ValueError` on any mismatch. Our check confirms both files are byte-identical to the supplied datasets and that the first 800 rows of v2 equal v1. History: 13 conventional-commit messages, from setup (`382578e`) through config (`1907a49`), features (`939f66c`), **training/tracking/baseline** (`c498836`), gate and CI (`47b47ff`), to the three PR branches. MLflow stamps every run with the git commit that was checked out when it ran (`aa11c19` for all runs in the database), so a result is tied to code, `dataset_version` (v1/v2) and the stored config.

Deviation we disclose: DVC was not used. The CSVs were committed to ordinary Git in `c129aaa` so that GitHub runners can train; this contradicts the brief's rule that data stay out of ordinary history. The tree at `c498836` contains the manifest but not the CSVs, so a full code+data restore is demonstrated from `c129aaa` onward (hash-verified); `c498836` needs the supplied file whose hash is in the manifest.

3 Configuration Management (Task 3)

Paths, seed, feature cut-offs (utilisation 0.80, delay 30 d), model family and parameters, tracking URI/experiment, split and all gate thresholds are YAML values validated by Pydantic V2 (type, range via `ge/le`, `Literal` enums). `load_config` fails immediately with the offending field; `test_deliberately_broken_schema_raises_informative_error` covers this. Each run logs the full config as a YAML artefact plus `feat_*/param_*` parameters.

Env.	Data	Model	CV folds	Test size	Gate: min PR / max drop PR,ROC / min recall	Register
dev	v1	logistic regression	3	0.25	0.50 / 0.02 / 0.65	no
ci	v1	XGBoost, 100 trees, depth 4	5	0.25	0.52 / 0.015 / 0.70	yes
prod	v2	XGBoost, 150 trees, depth 4	5	0.20	0.55 / 0.010 / 0.70	yes

Gaps: the tuning grid of the 12 runs lives in `scripts/run_experiments.py`, not in YAML (`search_type` is `fixed`), and `cv_folds/scoring_metric` are validated but not used by the training function.

4 Cleaning and Feature Engineering (Task 4)

Missing values. In v1, 24 rows (3.0%) have Credit_Limit, Current_Balance and Payment_Delay_Days missing *together* (72 cells; v2: 33 rows); their default rate (20.8%) is close to the rest (18.8%), so we keep the rows. Numeric columns use median and categoricals most-frequent imputation, both fitted *inside* the scikit-learn `Pipeline` on the training split only. The derived flag is 0 for these rows; imputing before deriving features changed mean PR-AUC of our logistic check over three seeds by only -0.009 ($0.8423 \rightarrow 0.8335$).

Feature (type)	Formula (B=balance, L=limit, M=spend, D=delay days, T=tenure)	Reason
Utilisation (ratio)	$U = \min(\max(B/L, 0), 2.5)$; if $L = 0$: $U = 2.5$ when $B > 0$ else 0	Share of the limit already used; clip and zero-limit rule avoid division by zero
Alert flag (binary)	$F = \mathbf{1}[D > 30 \vee U > 0.80]$	Thresholds from YAML; encodes classic delinquency/over-utilisation rules
Tenure stage (binned)	new ($T \leq 12$), established (13–36), mature (> 36)	Risk is non-linear in account age; one-hot encoded
Financial Stress Index (domain)	$S = \min(\max(M/L, 0), 2)\,(1 + \max(D, 0)/30)$	Spending burn rate relative to limit, amplified by payment lateness

Customer_ID is removed in `prepare_features_and_target`; the target is never an input (`test_no_target_leakage_and_id_exclusion`). The stateless feature creator and the fitted `ColumnTransformer` sit in one `Pipeline` that is saved with the model, so training and inference apply identical logic (`test_unseen_data_transformation_consistency`); a zero-limit edge case is tested.

5 Training, Tuning and Tracking (Task 5)

Baseline: logistic regression (C=1, balanced weights). Tuned families: random forest, XGBoost, LightGBM, three configurations each. All 12 runs log parameters, dataset version, config, metrics and model on the same stratified split (600 train / 200 test, 38 defaults). We reproduced the baseline exactly from the data (PR-AUC 0.7978, ROC 0.9215, recall 0.8947).

Run	PR	ROC	Rec.	Acc.	Brier	Run	PR	ROC	Rec.	Acc.	Brier
logreg C=1 (baseline)	.7978	.9215	.8947	.835	.1189	xgb default d3 (**champion**)	**.8091**	.9414	.7895	.895	.0777
logreg C=0.1	.7925	.9185	.8947	.835	.1215	xgb tuned d4, pos.weight	.7945	.9397	.8684	.870	.0992
logreg C=10	.7921	.9201	.8684	.830	.1195	xgb deep d5, lr 0.01	.7430	.8933	.6579	.890	.0840
rf 50 trees d3	.7539	.9170	.8947	.830	.1285	lgbm leaves 15	.7893	.9402	.7105	.880	.0918
rf 100 trees d5	.7498	.9193	.8158	.855	.0974	lgbm unbalanced, 31	.7949	.9404	.7895	.865	.0955
rf 200 trees d7	.7365	.9222	.8684	.860	.1033	lgbm tuned, 20	.7706	.9383	.8421	.865	.0983

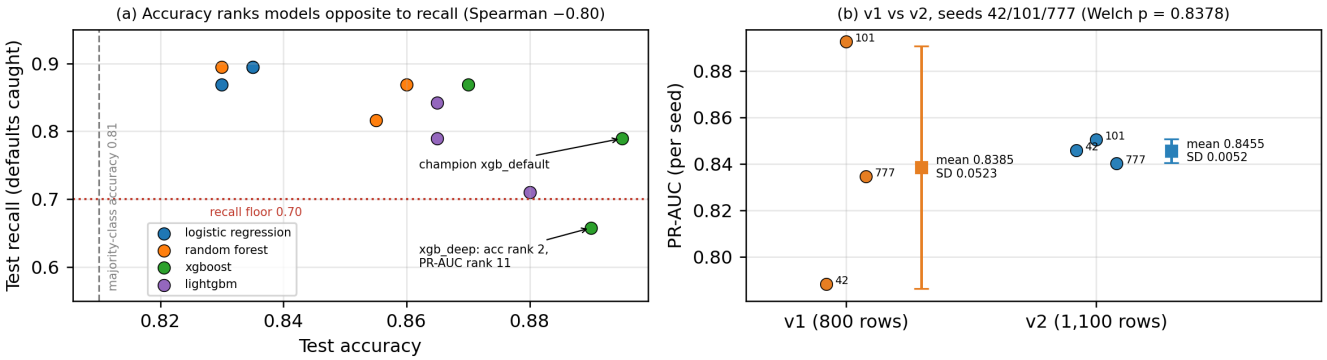


Figure 1: (a) The 12 runs: accuracy rewards models that miss defaults. (b) Task 7 seed-level PR-AUC, mean and ±1 SD.

Why not accuracy. Predicting "no default" for everyone already scores $1 - 0.19 = 0.81$ accuracy with zero recall; the logistic baseline (0.835) is only 2.5 points above that, whereas its PR-AUC of 0.7978 is 4.2× the no-skill level (prevalence 0.19). Across the 12 runs, rank correlation of accuracy with PR-AUC is $+0.12$ ($p = 0.72$) and with recall -0.80 ($p = 0.0017$): `xgb_deep` is 2nd by accuracy (0.890) but 11th by PR-AUC and misses 34% of defaults. Metrics: AP $= \sum_k (R_k - R_{k-1}) P_k$, Recall $= \frac{TP}{TP+FN}$, Brier $= \frac{1}{n}\sum(\hat{p}_i - y_i)^2$. **Selection:** highest PR-AUC ⇒ `xgb_default_depth3_lr0.1`, registered as `CreditRiskGateModel` v1 (MLflow run `dafca506`). Caveats: selection used the test split; PR-AUC gain over the baseline is $+0.011$ (inside noise); recall fell from 0.8947 to 0.7895 (30/38, exact 95% CI 0.627–0.904).

6 Automated Quality Gate (Task 6)

Workflow (`ci.yml`, on every pull request and push to main): checkout, Python 3.11, install, `pytest` (17 tests), train with `ci.yaml`, `python -m src.gate` (exit code 1 blocks the merge), and the Markdown comparison table is appended to the job summary so reviewers need not open logs. **Gate (Rule C):** accept iff

$$\text{PR} \geq \text{PR}_b - 0.015 \;\wedge\; \text{ROC} \geq \text{ROC}_b - 0.015 \;\wedge\; \text{Recall} \geq 0.70 \;\wedge\; \text{PR} \geq 0.52,$$

with baseline $\text{PR}_b = 0.7978$, $\text{ROC}_b = 0.9215$ (logistic regression, v1) from `baseline_metrics.json`.

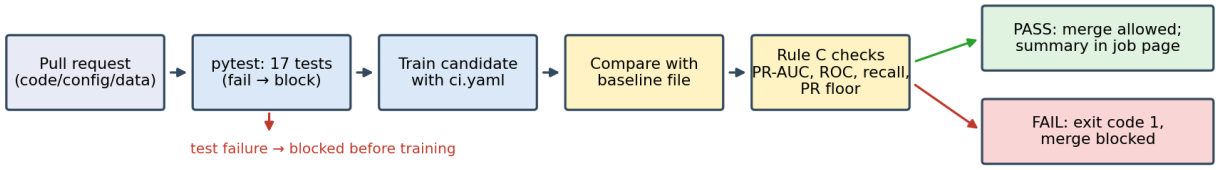


Figure 2: Gate workflow of `ci.yml`.

Branch / commit	Change	Outcome
PR #101 `00617e0`	Adds a comment line to `ci.yaml` (no behavioural change)	Candidate XGBoost (`ci.yaml`): PR 0.7883 ($\Delta = -0.0095$), ROC 0.9366, recall 0.8684 ⇒ **PASS** (inside tolerance; not an improvement)
PR #102 `deb5be4`	Trees 100→2, depth 4→1, lr 0.05→0.001	Quality regression ⇒ **BLOCK**. The demonstration script uses label-shuffled training (README: PR-AUC 0.4120, recall 0.45); we could not re-run XGBoost in our environment
PR #103 `9b85c5b`	Adds a test asserting $1 = 0$	`pytest` exits non-zero before training ⇒ **BLOCK**

Baseline-update rule (`governance_policy` in the baseline file): protected, needs approval by the risk-operations lead and a senior MLOps engineer, and the candidate must reach PR-AUC ≥ 0.7978 over ≥ 3 seeds with recall ≥ 0.70. *Gap:* `gate.py` checks a single seed with the 0.015 tolerance and the repository has no CODEOWNERS/branch-protection file, so the rule is documented but not technically enforced. Actions run links and timings are not part of the repository archive and are shown in the demonstration.

7 Data-Change Drill: v1 vs v2 (Task 7)

Same `ci.yaml` pipeline, seeds 42/101/777, each run tagged `dataset_version`.

Metric	v1 mean ± SD	v2 mean ± SD	Δ (v2−v1)	Welch p	Significant?
PR-AUC	0.8385 ± 0.0523	0.8455 ± 0.0052	$+0.0070$	0.8378	no
ROC-AUC	0.9484 ± 0.0141	0.9467 ± 0.0082	-0.0017	0.8679	no
Recall	0.7982 ± 0.0662	0.8013 ± 0.0588	$+0.0031$	0.9551	no
F1	0.7337 ± 0.0305	0.7177 ± 0.0235	-0.0160	0.5151	no

Reading. The $+0.007$ PR-AUC difference is one-seventh of the v1 seed SD; with $n = 3$ the Welch 95% CI of the difference is $[-0.121, +0.135]$ (df ≈ 2.0), so the study can only rule out very large effects and cannot show equivalence. The PR-AUC SD fell from 0.0523 to 0.0052, a **10-fold SD reduction (about 100-fold in variance)**; however Levene's test gives $p = 0.153$, so with three seeds this is suggestive, not proven. Further confounders: seeds change the split, and v1/v2 are scored on different test sets (200 vs. 275 rows). **Decision:** the v2-trained model passes the gate (PR $0.8455 \geq 0.7828$, ROC $0.9467 \geq 0.9065$, recall $0.8013 \geq 0.70$), so the gate accepts the retrained model; but we claim *no improvement* and keep `baseline_metrics.json` unchanged until baseline and candidate are compared on one fixed test set with at least three seeds. The extra 300 customers (+37.5%) are the reason to prefer v2 operationally.

8 Gate Threshold and Fairness Defense (Task 8)

Candidate rules on the 12 runs. A: ROC $\geq$ ROC$_b - 0.02$ (11/12 accepted; only `xgb_deep` rejected). B: PR $\geq \max(\text{PR}_b, 0.80)$ (1/12: only the champion; it would even reject the production baseline, 0.7978). C: Rule above (7/12: three logistic, `xgb_default`, `xgb_tuned`, `lgbm_default`, `lgbm_unbalanced`; rejects all three random forests, whose PR-AUC is 0.044–0.061 below baseline, `lgbm_tuned` (-0.027) and `xgb_deep`). Rule A is blind to the minority class: it admits a forest with PR-AUC 0.7365 and a model with recall 0.7105. B is so tight that it blocks equivalent models.

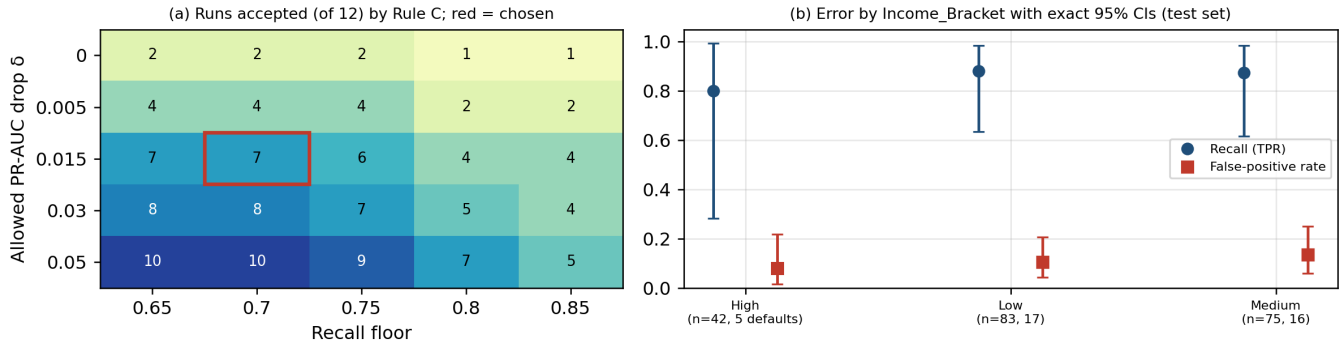


Figure 3: (a) Sensitivity of Rule C to its two parameters. (b) Subgroup recall and false-positive rate with exact 95% intervals.

Income_Bracket	Test n	Defaults	Actual rate	Predicted rate	Recall	FPR	Precision
High	42	5	0.119	0.167	0.800 (4/5)	0.081 (3/37)	0.571
Low	83	17	0.205	0.265	0.882 (15/17)	0.106 (7/66)	0.682
Medium	75	16	0.213	0.293	0.875 (14/16)	0.136 (8/59)	0.636

Fairness. Recall is 0.80–0.88 and FPR 8–14% in all brackets; the model over-flags every group by a similar factor (1.29–1.40×). Fisher's test finds no difference (FPR High vs. Medium $p = 0.52$; recall High vs. Low $p = 1.0$), but the High bracket has only 5 defaults (recall CI 0.28–0.99), so this is *absence of evidence*. We accept the disparity as non-actionable for merging, not as a fairness clearance. The audit used the `ci.yaml` XGBoost (registry v2, `fairness_audit_run`); the champion v1 was not audited.

Recommendation and trade-off. Adopt **Rule C**: it is the only rule that guards both ranking quality (PR-AUC) and the business-critical recall floor while tolerating small noise. *We accept* (a) a 0.015 tolerance that is smaller than the seed SD (0.0523), so a single-seed gate can pass a genuinely slightly worse model or fail an equal one, and (b) a recall floor (0.70) that is itself within the CI of a 38-default test set. Sensitivity (Fig. 3a): $\delta = 0.005$ would accept only 4 runs, $\delta = 0.03$ 8 runs; recall floor 0.80 would accept 4. **Evidence that would change our recommendation:** multi-seed baseline and candidate on one fixed, larger test set (seed SD below 0.015 $\Rightarrow$ keep; above $\Rightarrow$ gate on 3-seed mean), ≥ 30 defaults per bracket showing a recall gap above 0.10 with an interval excluding 0, or a bank-supplied cost ratio for missed versus falsely flagged customers.

9 Limitations Found in Our Own Work

(1) Model selection and gating use one 200-row test split with no validation set, so selected-model metrics are optimistic. (2) Data are in ordinary Git; DVC would meet the brief. (3) Baseline-update protection and the 3-seed rule are not enforced in code. (4) Tuning grids and the unused `cv_folds` are not YAML-driven. (5) The fairness audit covers the `ci.yaml` model, not champion v1; MLflow stage fields are unused. (6) `baseline_metrics.json` records `git_commit: c51e19d`, whereas the file

entered Git in `c498836`. (7) Verification scope: we independently recomputed data statistics, the baseline logistic metrics, Welch tests, intervals and rule tables from the supplied data; XGBoost, LightGBM and random-forest figures and the PR #102 results are taken from the repository's MLflow and CSV records and were not re-executed by the report's drafter. (8) Our random-forest re-run differed from the recorded run (PR-AUC 0.7877 vs. 0.7539), consistent with library-version sensitivity.

10 Academic Integrity Disclosure

Datasets are the supplied files (byte-identical, hashes match the manifest); only free, open-source tools are used; no credentials are committed. The repository README discloses use of AI assistants (Anthropic Claude, Google Gemini, OpenAI) for syntax help. **This report text was drafted with Claude (Anthropic) from the repository, MLflow database and CSV artefacts; Claude also independently recomputed the figures listed in Sec. 9.** The brief forbids submitting AI-generated reports and defense memos as one's own, so the authors state this use openly and remain responsible for every claim; both partners will explain and modify the code and the defense reasoning live in the demonstration.